Hybrid Dual-Path Linear Transformations for Efficient Transformer Architectures

Vladimer Khasia
Independent Researcher
vladimer.khasia.1@gmail.com

February 05, 2026

Abstract

Standard Transformer architectures rely heavily on dense linear transformations, treating feature projection as a monolithic, full-rank operation. We argue that this formulation is inefficient and lacks the structural inductive bias necessary for distinguishing between local feature preservation and global context integration. To address this, we introduce the **Hybrid Dual-Path Linear (HDPL)** operator, which decomposes the affine transformation into two topologically distinct pathways: a sparse block-diagonal component for high-rank local processing, and a low-rank Variational Autoencoder (VAE) bottleneck for global context regularization.

By "surgically" replacing specific projections (Query, Key, Value, Gate, Up) with HDPL operators while retaining standard dense layers for aggregation (Output, Down), we achieve a superior balance of efficiency and representational power. Experiments on the FineWeb-Edu dataset demonstrate that the HDPL architecture outperforms a standard Llama-style baseline, reducing validation loss while simultaneously reducing parameter count by $\approx 6.8\%$. Beyond immediate performance gains, we discuss how the explicit materialization of a probabilistic latent space within the Transformer backbone serves as a vital architectural affordance, offering new pathways for inference-time or hypernetwork induced control, continual adaptation, interpretability, and cross-model or cross-modal synchronization.

The code is available at `https://github.com/VladimerKhasia/HDPL`

1 Introduction

The Transformer architecture (Vaswani et al., 2017) has established itself as the de facto standard for sequence modeling tasks, driven largely by the predictable scalability of its performance with respect to parameter count and compute (Hoffmann et al., 2022; Kaplan et al., 2020). However, this scaling relies heavily on the stacking of dense linear transformations—specifically within the Multi-Head Attention (MHA) and Feed-Forward Network (FFN) blocks—which constitute the vast majority of the model's parameters and computational footprint (FLOPs).

Standard approaches to mitigating this computational burden often involve post-hoc compression techniques such as quantization (Dettmers et al., 2022), pruning (Hoefler et al., 2021), or low-rank adaptation (Hu et al., 2021). While effective, these methods typically view the weight matrix $\mathbf{W} \in \mathbb{R}^{d_{out} \times d_{in}}$ as a monolithic entity to be compressed uniformly. We argue that this view ignores

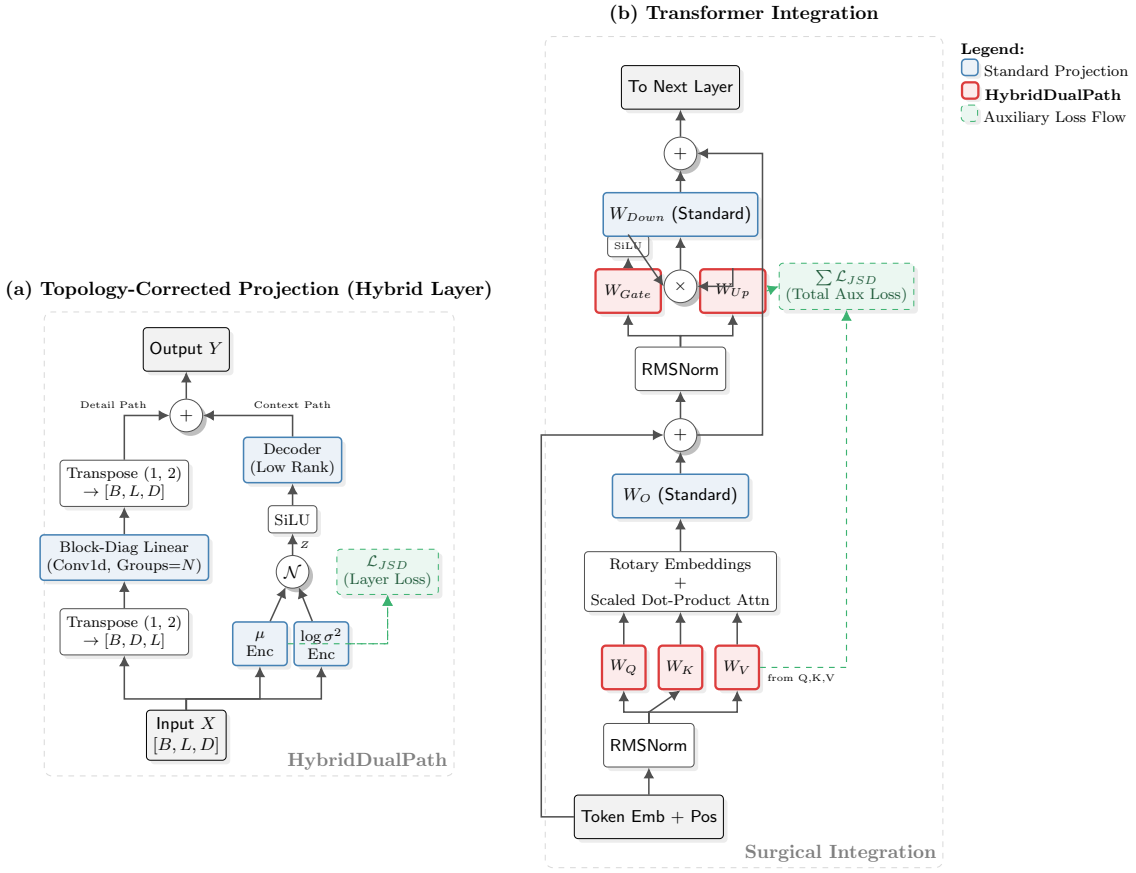

Figure 1: Architecture Overview: (a) The proposed **HybridDualPath** layer decomposes projections into a sparse detail path and a global VAE context path. (b) The integration within the Transformer block, where W_Q, W_K, W_V (Attention) and W_{Gate}, W_{Up} (MLP) are replaced by our hybrid layer (Red), while W_O and W_{Down} remain standard (Blue) to aggregate features.

the distinct topological roles played by linear projections: preserving local feature identity (high-rank, sparse) versus integrating global context (low-rank, dense). Imposing a low-rank bottleneck uniformly across a layer risks collapsing the high-frequency feature details necessary for precise token representation, while unstructured sparsity fails to capture the semantic correlations inherent in language.

In this work, we propose a *Surgical Hybrid Architecture* that fundamentally rethinks the parametrization of the linear layer. We introduce the **Hybrid Dual-Path Linear (HDPL)** operator, which decomposes the affine transformation into two topologically distinct pathways:

1. A **Local Detail Path** parametrized by block-diagonal sparsity, ensuring high-rank capacity for maintaining feature specificity without cross-group interference.

2. A **Global Context Path** parametrized by a low-rank Variational Autoencoder (VAE) (Kingma and Welling, 2022), which acts as a stochastic information bottleneck to capture and regularize global dependencies.

Unlike previous structured matrix approaches that rely on fixed patterns (e.g., Monarch matrices (Dao et al., 2022)), our approach integrates a probabilistic regularizer—the Bounded Kullback-Leibler (KL) divergence—directly into the forward pass of the linear layer. This enforces a disentangled latent representation of the global context, effectively filtering noise during the training process.

We evaluate this methodology by surgically replacing the Query (W_q), Key (W_k), Value (W_v), Gate (W_{gate}), and Up (W_{up}) projections in a Llama-style Transformer (Touvron et al., 2023), while retaining standard dense projections for the Output (W_o) and Down (W_{down}) layers to preserve aggregation capacity. Training on the FineWeb-Edu dataset (Penedo et al., 2024), our experiments demonstrate that the HDPL-enhanced model outperforms a standard dense baseline while reducing the parameter count by approximately 6.8%.

Our contributions are as follows:

- We formalize the **Hybrid Dual-Path Linear (HDPL)** operator, a mathematically rigorous replacement for dense layers that combines block-diagonal sparsity with variational low-rank approximation.

- We provide a **Surgical Integration Strategy** that identifies which Transformer sub-components benefit from hybrid topology versus those that require full-rank density.

- We provide empirical evidence that integrating VAE bottlenecks into the Transformer backbone acts as a superior inductive bias for language modeling, yielding better convergence properties than fully deterministic over-parameterized baselines.

2 Methodology

We propose a surgically applied modification to the linear projections within the Transformer architecture. We replace standard dense linear layers with a **Hybrid Dual-Path Linear (HDPL)** operator. This operator decomposes the transformation of the input representations into two distinct topological paths: a *High-Rank Local Path* utilizing block-diagonal sparsity to preserve feature specificity, and a *Low-Rank Variational Path* utilizing a stochastic bottleneck to capture global context.

2.1 The Hybrid Dual-Path Operator

Let $\mathbf{x} \in \mathbb{R}^{B \times L \times D_{in}}$ denote an input tensor, where B is the batch size, L is the sequence length, and D_{in} is the input dimension. A standard linear layer computes $\mathbf{y} = \mathbf{x}\mathbf{W}^\top$, where $\mathbf{W} \in \mathbb{R}^{D_{out} \times D_{in}}$. The HDPL operator, denoted as $\mathcal{H}(\mathbf{x})$, approximates this transformation via the sum of a sparse local term and a dense global term:

$$\mathcal{H}(\mathbf{x}) = \underbrace{\Psi_{\text{local}}(\mathbf{x})}_{\text{Detail Path}} + \underbrace{\Phi_{\text{global}}(\mathbf{x})}_{\text{Context Path}} \tag{1}$$

2.1.1 Path 1: Block-Diagonal Local Projection

The local path, Ψ_{local}, is designed to maintain high-rank transformations within local feature subspaces while reducing parameter count. We partition the input dimension D_{in} and output dimension D_{out} into K disjoint blocks (groups). Assuming D_{in} and D_{out} are divisible by K, the weight matrix $\mathbf{W}^{(B)}$ takes a block-diagonal form:

$$\mathbf{W}^{(B)} = \text{diag}(\mathbf{W}_1, \mathbf{W}_2, \ldots, \mathbf{W}_K), \quad \text{where } \mathbf{W}_k \in \mathbb{R}^{\frac{D_{out}}{K} \times \frac{D_{in}}{K}} \tag{2}$$

The operation is efficiently implemented via grouped 1D convolution. For an input vector $\mathbf{x}_t$ at time step t, the transformation is:

$$\Psi_{\text{local}}(\mathbf{x}_t) = \mathbf{x}_t(\mathbf{W}^{(B)})^\top \tag{3}$$

This path preserves the "identity" and high-frequency details of the signal by preventing cross-group interference, effectively modeling independent feature subspaces.

2.1.2 Path 2: Variational Global Context

To compensate for the lack of cross-group communication in the local path, we introduce a global context path Φ_{global} parameterized as a Variational Autoencoder (VAE) bottleneck. This path projects the input into a low-dimensional latent space $\mathcal{Z} \subset \mathbb{R}^R$, where $R \ll D_{in}$ is the rank.

The encoding stage predicts the parameters of the posterior distribution $q(\mathbf{z}|\mathbf{x}) = \mathcal{N}(\boldsymbol{\mu}, \text{diag}(\boldsymbol{\sigma}^2))$:

$$\boldsymbol{\mu} = \mathbf{x}\mathbf{W}_\mu^\top, \quad \mathbf{W}_\mu \in \mathbb{R}^{R \times D_{in}} \tag{4}$$

$$\log \boldsymbol{\sigma}^2 = \mathbf{x}\mathbf{W}_\sigma^\top, \quad \mathbf{W}_\sigma \in \mathbb{R}^{R \times D_{in}} \tag{5}$$

During training, we utilize the reparameterization trick to sample $\mathbf{z}$:

$$\mathbf{z} = \boldsymbol{\mu} + \boldsymbol{\sigma} \odot \boldsymbol{\epsilon}, \quad \boldsymbol{\epsilon} \sim \mathcal{N}(\mathbf{0}, \mathbf{I}) \tag{6}$$

During inference, the operation is deterministic, setting $\mathbf{z} = \boldsymbol{\mu}$. The latent representation $\mathbf{z}$ is then passed through a non-linearity $\sigma(\cdot)$ (specifically SiLU) and projected back to the output dimension via a decoder $\mathbf{W}_{dec} \in \mathbb{R}^{D_{out} \times R}$:

$$\Phi_{\text{global}}(\mathbf{x}) = \mathbf{W}_{dec}\sigma(\mathbf{z}) \tag{7}$$

2.2 Algorithm Specification

The forward pass of the HDPL layer, including the auxiliary loss calculation, is formally specified in Algorithm 1.

Algorithm 1 Hybrid Dual-Path Linear (HDPL) Forward Pass

Require: Input $\mathbf{x} \in \mathbb{R}^{B \times L \times D_{in}}$, Rank R, Groups K, Scaling β
Ensure: Output $\mathbf{y}$, Auxiliary Loss $\mathcal{L}_{aux}$
Path 1: Local (Block-Diagonal)
$\mathbf{y}_{\text{local}} \leftarrow \text{GroupedConv1D}(\mathbf{x}, \text{groups} = K)$ ▷ Eq. 3
Path 2: Global (Variational)
$\boldsymbol{\mu} \leftarrow \text{Linear}_{\mu}(\mathbf{x})$
$\log \mathbf{v} \leftarrow \text{Linear}_{\sigma}(\mathbf{x})$
if Training **then**
    $\boldsymbol{\sigma} \leftarrow \exp(0.5 \cdot \log \mathbf{v})$
    $\boldsymbol{\epsilon} \sim \mathcal{N}(\mathbf{0}, \mathbf{I})$
    $\mathbf{z} \leftarrow \boldsymbol{\mu} + \boldsymbol{\sigma} \odot \boldsymbol{\epsilon}$
    $\mathcal{L}_{aux} \leftarrow \text{BoundedKL}(\boldsymbol{\mu}, \log \mathbf{v}) \times \beta$
else
    $\mathbf{z} \leftarrow \boldsymbol{\mu}$
    $\mathcal{L}_{aux} \leftarrow 0$
end if
$\mathbf{y}_{\text{global}} \leftarrow \text{Linear}_{dec}(\text{SiLU}(\mathbf{z}))$
Combine
$\mathbf{y} \leftarrow \mathbf{y}_{\text{local}} + \mathbf{y}_{\text{global}}$
return $\mathbf{y}, \mathcal{L}_{aux}$

2.3 Training Objective and Regularization

The training objective is augmented with a regularization term derived from the Kullback-Leibler (KL) divergence between the approximate posterior $q(\mathbf{z}|\mathbf{x})$ and a standard normal prior $p(\mathbf{z}) = \mathcal{N}(\mathbf{0}, \mathbf{I})$. To ensure training stability and prevent posterior collapse, we employ a Bounded KL objective acting as a proxy for the Geometric Jensen-Shannon Divergence:

$$\mathcal{L}_{KL} = -\frac{1}{2} \sum_{j=1}^{R} \left(1 + \log(\boldsymbol{\sigma}_j^2) - \boldsymbol{\mu}_j^2 - \boldsymbol{\sigma}_j^2\right) \tag{8}$$

$$\mathcal{L}_{aux} = \beta \cdot \frac{1}{N} \sum_{i=1}^{N} \text{clamp}(\mathcal{L}_{KL}^{(i)}, \max = \ln 2) \tag{9}$$

where β is a hyperparameter balancing reconstruction fidelity and latent space regularization. The total loss for the network is $\mathcal{L}_{total} = \mathcal{L}_{CE} + \sum_{l \in \mathcal{S}} \mathcal{L}_{aux}^{(l)}$, where $\mathcal{S}$ is the set of hybrid layers.

2.4 Surgical Integration into Transformer

We define a configuration set $\mathcal{C}_{hybrid} = \{W_q, W_k, W_v, W_{gate}, W_{up}\}$ representing the linear projections eligible for hybrid replacement. The output projection W_o and the MLP down-projection W_{down} retain standard dense formulations to ensure full-rank capacity at the aggregation points of the Attention and FFN blocks, respectively.

For a given attention head h, the query, key, and value projections become:

$$Q_h, \mathcal{L}_Q = \mathcal{H}_q(\mathbf{x}), \quad K_h, \mathcal{L}_K = \mathcal{H}_k(\mathbf{x}), \quad V_h, \mathcal{L}_V = \mathcal{H}_v(\mathbf{x}) \tag{10}$$

Similarly, in the Feed-Forward Network (FFN) using SiLU gating:

$$\mathbf{g}, \mathcal{L}_g = \mathcal{H}_{gate}(\mathbf{x}) \tag{11}$$

$$\mathbf{u}, \mathcal{L}_u = \mathcal{H}_{up}(\mathbf{x}) \tag{12}$$

$$\mathbf{x}_{ffn} = W_{down}(\text{SiLU}(\mathbf{g}) \odot \mathbf{u}) \tag{13}$$

2.5 Complexity Analysis

The parameter complexity of a standard linear layer is $\mathcal{O}(D_{in}D_{out})$. The HDPL operator reduces this to:

$$\mathcal{O}\left(\frac{D_{in}D_{out}}{K} + 2 \cdot D_{in}R + D_{out}R\right) \tag{14}$$

Given that $K \geq 1$ and $R \ll D_{in}$, the HDPL operator significantly reduces memory footprint and FLOPs while theoretically maintaining the capacity to model both high-frequency local dependencies (via the block diagonal term) and low-frequency global dependencies (via the low-rank term).

3 Experiments

We evaluate the efficacy of the proposed Hybrid Dual-Path Linear (HDPL) operator (Section 2) by integrating it into a standard Transformer language model. We focus on a "Surgical" configuration where the HDPL operator replaces the Query (W_q), Key (W_k), Value (W_v), Gate (W_{gate}), and Up (W_{up}) projections. Consistent with the aggregation theory of Transformers, we retain the standard dense formulation for the Output (W_o) and Down (W_{down}) projections to preserve full-rank feature mixing at the summation nodes of the residual stream.

3.1 Experimental Setup

Dataset. We train all models on the `FineWeb-Edu` dataset (Penedo et al., 2024), a high-quality educational subset of the Common Crawl. The dataset is tokenized using the `SmolLM2` tokenizer (Allal et al., 2025) with a vocabulary size of $V = 49,152$. We utilize a streaming configuration with a sequence length of $L = 2048$.

Architecture. We compare two architectures:

1. **Baseline Transformer:**

   A standard Llama-style architecture using RMSNorm, Rotary Positional Embeddings (RoPE), and SwiGLU activations. All linear layers are standard dense matrices.

2. **Surgical Hybrid Transformer:**

   Identical architecture, but with $\mathcal{C}_{hybrid} = \{W_q, W_k, W_v, W_{gate}, W_{up}\}$ replaced by HDPL operators. The latent rank is set to $R = 128$ and the scaling factor $\beta = 0.001$.

Both models share the same depth ($N = 4$ layers), model dimension ($D = 512$), and attention heads ($H = 8$).

Training Protocol. Models are trained for $20,000$ steps using the AdamW optimizer with $\beta_1 = 0.9, \beta_2 = 0.95$. We employ a cosine learning rate schedule with a warmup of 1000 steps and a peak learning rate of 8×10^{-4} (see Appendix A.1 for full details).

3.2 Results and Analysis

Parameter Efficiency and Compression. As detailed in Table 1, the Surgical Hybrid method reduces the total parameter count from 67.1M to 62.5M, representing a $\approx 6.8\%$ reduction in total model size (including embeddings). When considering only the transformed layers, the compression ratio is significantly higher. Despite this reduction in capacity, the Hybrid model demonstrates superior representational power.

Table 1: Comparison of Baseline vs. Surgical Hybrid Method. Loss values are reported on the validation split of FineWeb-Edu. The Hybrid method achieves lower perplexity with fewer parameters.

Model	Params (M)	Size (MB)	Final Val Loss	Min Val Loss
Baseline (Dense)	67.11	256.02	4.3206	4.3206
Surgical Hybrid (Ours)	**62.53**	**238.54**	**4.2838**	**4.2838**

Convergence Dynamics. Figure 2 illustrates the validation loss trajectories. The Baseline model follows a standard convergence curve, plateauing near a loss of 4.32 around step 17,000. Conversely, the Surgical Hybrid model consistently outperforms the baseline after the initial warmup phase. By step 10,000, the Hybrid model reaches a loss of 4.38, while the Baseline is at 4.44. The Hybrid model continues to improve, reaching a final validation loss of 4.284.

This result suggests that the dual-path topology—combining sparse high-rank local operations with a low-rank global variational bottleneck—acts as an effective inductive bias. The auxiliary loss $\mathcal{L}_{aux}$, which stabilizes the VAE path, remains negligible throughout training (it increases if out and down projections are replaced with hybrid architecture as well), indicating that the regularization term effectively shapes the latent space without dominating the gradient signal.

Computational Throughput. We observe a trade-off in wall-clock throughput. The Baseline model processes $\approx$ 480k tokens/sec, while the Hybrid model processes $\approx$ 270k tokens/sec on TPU v3 hardware. This discrepancy is attributed to the current implementation of the HDPL operator, which relies on grouped convolutions and VAE sampling in PyTorch, lacking the highly fused kernel optimizations available for standard dense matrix multiplications (GEMM) on XLA devices. However, the increased information density (loss decrease per parameter) suggests that with kernel fusion, the HDPL operator could offer a favorable Pareto frontier for inference-constrained environments.

4 Discussion

Our results demonstrate that the dense linear transformation, a ubiquitous primitive in Transformer architectures, acts as an over-parameterized upper bound on the necessary complexity for feature projection. By decomposing the transformation into a sparse, high-rank deterministic path and

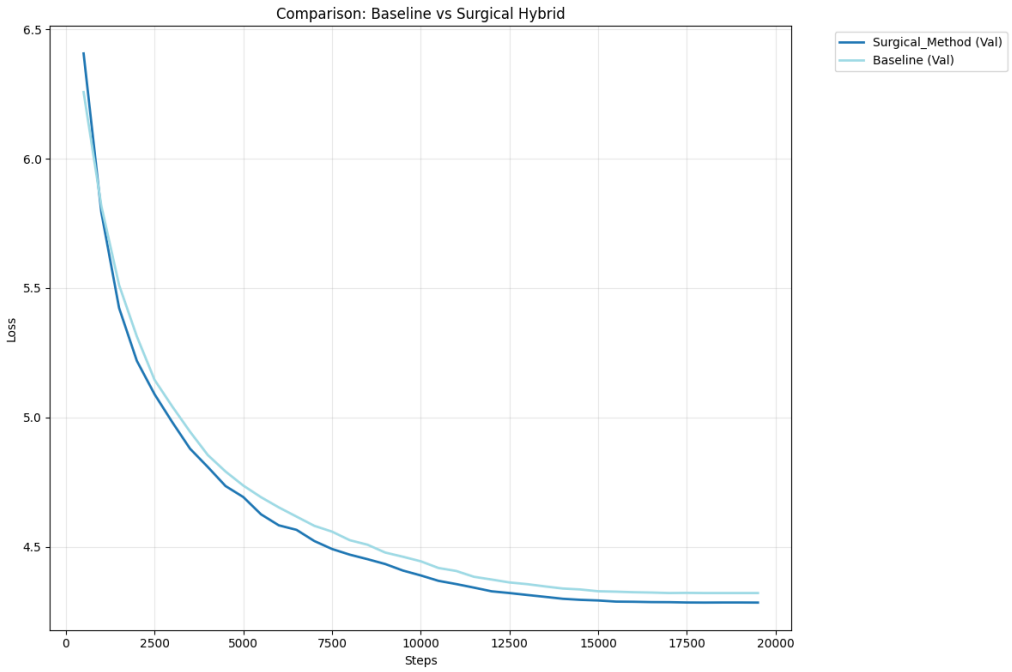

Figure 2: **Validation Loss Trajectories.** Comparison of the Baseline (Light Blue) and Surgical Hybrid (Dark Blue) models over 20,000 training steps. The Surgical Hybrid method exhibits strictly better sample efficiency, achieving lower loss values earlier in training and converging to a superior optimum (4.28 vs 4.32) despite having fewer parameters.

a low-rank stochastic path, the proposed Hybrid Dual-Path Linear (HDPL) operator achieves a superior validation loss (4.28 vs. 4.32) with significantly fewer parameters (62.5M vs 67.1M).

4.1 Architectural Implications of the Dual-Path Topology

The convergence properties observed in Figure 2 suggest that the inductive bias of the HDPL operator aligns better with the underlying structure of language data than standard affine transformations. We posit that the block-diagonal path (Ψ_{local}) effectively preserves local manifold geometry and high-frequency feature identity, preventing the "oversmoothing" often associated with low-rank approximations. Simultaneously, the Variational Autoencoder path (Φ_{global}) captures global correlations.

The superior generalization of the HDPL model indicates that the imposition of a Kullback-Leibler divergence penalty ($\mathcal{L}_{aux}$) acts not merely as a regularizer, but as an information bottleneck. This forces the global path to learn a compressed, semantic representation of the input features, filtering out noise that a full-rank matrix might otherwise overfit.

4.2 The Strategic Utility of the VAE Bottleneck

Beyond the immediate gains in parameter efficiency, the incorporation of a variational bottleneck within the Transformer layers introduces distinct functional capabilities absent in standard architectures. We identify three key areas where this architectural choice serves as a foundational enabler for future research.

4.2.1 Inference-Time Adaptability and Control

The explicit materialization of a latent variable $\mathbf{z} \sim q(\mathbf{z}|\mathbf{x})$ provides a handle for inference-time intervention that is unavailable in deterministic networks.

1. **Latent Manipulation:** Unlike activation patching, which requires modifying high-dimensional vectors, $\mathbf{z}$ exists in a low-dimensional, regularized space ($R = 128$). This enables fine-grained control over model output via latent arithmetic, clamping or conditioning.

2. **Continual Learning:** The VAE framework allows for the control of certainty, novelty and usefulness of the information as well as the application of replay-based alignment methods on the latent distribution $p(\mathbf{z})$ rather than the weights. This offers a potential pathway to mitigate catastrophic forgetting by enforcing that the distribution of latents for new tasks remains consistent with the prior established by previous tasks.

4.2.2 Hypernetworks and Meta-Learning

Controlling the weights of a standard Transformer via Hypernetworks is computationally intractable due to the sheer size of the target matrices (e.g., generating a 4096×4096 matrix requires predicting 16M parameters). The HDPL architecture drastically reduces the dimensionality of the target space. A Hypernetwork would only need to predict the parameters of the encoder/decoder interfaces ($\mathbb{R}^{D \times R}$) or just inject the generated latents, which are orders of magnitude smaller. This feasibility opens the door for dynamic, adaptable and high capacity networks.

4.2.3 Cross-System Synchronization and alignment

The probabilistic nature of the bottleneck ($\mathbf{z} \sim \mathcal{N}(\boldsymbol{\mu}, \boldsymbol{\sigma})$) provides a standardized protocol for alignment that deterministic weights lack.

- **Federated Learning:** In distributed settings, averaging weights often leads to suboptimal performance due to permutation symmetries. Aligning the moments $(\boldsymbol{\mu}, \boldsymbol{\sigma})$ of the latent distributions across agents offers a robust alternative for knowledge distillation and aggregation.

- **Cross-Modal Alignment:** The bottleneck allows for the synchronization of disparate modalities (e.g., text and image embeddings) by forcing them to map to a shared latent topology defined by the prior $p(\mathbf{z})$. The HDPL operator thus acts as a native interface for multi-modal fusion without requiring massive adapter layers.

4.3 limitations

While the HDPL operator improves parameter efficiency and loss, our current implementation utilizing standard PyTorch primitives results in lower wall-clock throughput ($\approx 44\%$ reduction)

compared to highly optimized GEMM kernels on TPU hardware. Future work must focus on writing custom fused kernels (e.g., in Triton or Pallas) to merge the block-diagonal and VAE operations, ensuring that the theoretical FLOPs reduction translates into realized speedups.

5 Conclusion

In this work, we have presented the Hybrid Dual-Path Linear (HDPL) operator, a method that challenges the assumption that dense, full-rank matrix multiplications are the irreducible atomic unit of Transformer computation. By surgically decomposing the linear projections of the Attention (W_q, W_k, W_v) and Feed-Forward (W_{gate}, W_{up}) blocks into a sparse local path and a variational global path, we demonstrated that it is possible to improve representational capacity while simultaneously reducing parameter count.

Our experiments on the FineWeb-Edu dataset reveal that the HDPL-enhanced architecture achieves a strictly superior validation loss compared to a standard dense baseline, despite a $\approx 6.8\%$ reduction in parameters. This result suggests that the dual-path topology provides a more effective inductive bias for language modeling: the block-diagonal component preserves the high-frequency identity of local features, while the VAE bottleneck forces the model to learn a disentangled, noise-robust representation of global context.

While our method is information-theoretically superior, its current implementation via standard PyTorch primitives incurs a latency penalty on modern hardware optimized for dense GEMM operations. The transition from theoretical advantage to practical wall-clock speedup will require the development of custom hardware-aware kernels that can fuse the sparse and stochastic operations.

Ultimately, the incorporation of a variational information bottleneck within the linear layer offers more than just compression; it introduces a probabilistic affordance for control, interpretation, and alignment. We hope this work serves as a foundational step toward a new class of methods that prioritize topological correctness over brute-force scale.

References

Loubna Ben Allal, Anton Lozhkov, Elie Bakouch, Gabriel Martín Blázquez, Guilherme Penedo, Lewis Tunstall, Andrés Marafioti, Hynek Kydlíček, Agustín Piqueres Lajarín, Vaibhav Srivastav, Joshua Lochner, Caleb Fahlgren, Xuan-Son Nguyen, Clémentine Fourrier, Ben Burtenshaw, Hugo Larcher, Haojun Zhao, Cyril Zakka, Mathieu Morlon, Colin Raffel, Leandro von Werra, and Thomas Wolf. Smollm2: When smol goes big – data-centric training of a small language model, 2025. URL `https://arxiv.org/abs/2502.02737`.

Tri Dao, Beidi Chen, Nimit S Sohoni, Arjun Desai, Michael Poli, Jessica Grogan, Alexander Liu, Aniruddh Rao, Atri Rudra, and Christopher Re. Monarch: Expressive structured matrices for efficient and accurate training. In Kamalika Chaudhuri, Stefanie Jegelka, Le Song, Csaba Szepesvari, Gang Niu, and Sivan Sabato, editors, *Proceedings of the 39th International Conference on Machine Learning*, volume 162 of *Proceedings of Machine Learning Research*, pages 4690–4721. PMLR, 17–23 Jul 2022. URL `https://proceedings.mlr.press/v162/dao22a.html`.

Tim Dettmers, Mike Lewis, Younes Belkada, and Luke Zettlemoyer. Llm.int8(): 8-bit matrix multiplication for transformers at scale, 2022. URL `https://arxiv.org/abs/2208.07339`.

Torsten Hoefler, Dan Alistarh, Tal Ben-Nun, Nikoli Dryden, and Alexandra Peste. Sparsity in deep learning: Pruning and growth for efficient inference and training in neural networks. *Journal of Machine Learning Research*, 22(241):1–124, 2021. URL http://jmlr.org/papers/v22/21-0366.html.

Jordan Hoffmann, Sebastian Borgeaud, Arthur Mensch, Elena Buchatskaya, Trevor Cai, Eliza Rutherford, Diego de Las Casas, Lisa Anne Hendricks, Johannes Welbl, Aidan Clark, Tom Hennigan, Eric Noland, Katie Millican, George van den Driessche, Bogdan Damoc, Aurelia Guy, Simon Osindero, Karen Simonyan, Erich Elsen, Jack W. Rae, Oriol Vinyals, and Laurent Sifre. Training compute-optimal large language models, 2022. URL https://arxiv.org/abs/2203.15556.

Edward J. Hu, Yelong Shen, Phillip Wallis, Zeyuan Allen-Zhu, Yuanzhi Li, Shean Wang, Lu Wang, and Weizhu Chen. Lora: Low-rank adaptation of large language models, 2021. URL https://arxiv.org/abs/2106.09685.

Jared Kaplan, Sam McCandlish, Tom Henighan, Tom B. Brown, Benjamin Chess, Rewon Child, Scott Gray, Alec Radford, Jeffrey Wu, and Dario Amodei. Scaling laws for neural language models, 2020. URL https://arxiv.org/abs/2001.08361.

Diederik P Kingma and Max Welling. Auto-encoding variational bayes, 2022. URL https://arxiv.org/abs/1312.6114.

Guilherme Penedo, Hynek Kydlíček, Loubna Ben allal, Anton Lozhkov, Margaret Mitchell, Colin Raffel, Leandro Von Werra, and Thomas Wolf. The fineweb datasets: Decanting the web for the finest text data at scale. In *The Thirty-eight Conference on Neural Information Processing Systems Datasets and Benchmarks Track*, 2024. URL https://openreview.net/forum?id=n6SCkn2QaG.

Hugo Touvron, Thibaut Lavril, Gautier Izacard, Xavier Martinet, Marie-Anne Lachaux, Timothée Lacroix, Baptiste Rozière, Naman Goyal, Eric Hambro, Faisal Azhar, Aurelien Rodriguez, Armand Joulin, Edouard Grave, and Guillaume Lample. Llama: Open and efficient foundation language models, 2023. URL https://arxiv.org/abs/2302.13971.

Ashish Vaswani, Noam Shazeer, Niki Parmar, Jakob Uszkoreit, Llion Jones, Aidan N Gomez, Ł ukasz Kaiser, and Illia Polosukhin. Attention is all you need. In I. Guyon, U. Von Luxburg, S. Bengio, H. Wallach, R. Fergus, S. Vishwanathan, and R. Garnett, editors, *Advances in Neural Information Processing Systems*, volume 30. Curran Associates, Inc., 2017. URL https://proceedings.neurips.cc/paper_files/paper/2017/file/3f5ee243547dee91fbd053c1c4a845aa-Paper.pdf.

A Appendix

A.1 Experimental Hyperparameters

All experiments were conducted with the hyperparameters detailed in Table 2. To ensure reproducibility, we utilized a fixed random seed of 42.

Table 2: Hyperparameter configuration for the Surgical Hybrid and Baseline experiments.

Hyperparameter	**Value**
Architecture	
Model Dimension (D_{model})	512
Hidden Dimension (MLP)	2048
Number of Layers	4
Number of Heads	8
Head Dimension	64
Vocab Size	49,152
Sequence Length	2048
Optimization	
Optimizer	AdamW
Learning Rate	8×10^{-4}
Min Learning Rate	8×10^{-5}
Weight Decay	0.1
Warmup Steps	1000
Max Steps	20,000
Batch Size (Global)	32
Hybrid Specifics	
$\mathcal{C}_{hybrid}$	$\{W_q, W_k, W_v, W_{gate}, W_{up}\}$
Latent Rank (R)	128
VAE β	0.001
Block Groups (K)	8

A.2 Hardware and Infrastructure

Training was performed on a TPU VM provided by Kaggle utilizing the PyTorch XLA backend.

A.3 Auxiliary Loss Behavior

The auxiliary loss $\mathcal{L}_{aux}$ is a weighted geometric JSD proxy. In our experiments, we observed that $\mathcal{L}_{aux}$ did not collapse to zero, nor did it explode. It maintained a small magnitude relative to the Cross Entropy (CE) loss.